Dynamic Programming

- notice any recursive subproblems
- decide what to solve inside input
- think recursively to use recursion
- write memoization to use memoization
- draw a strategy tree !!!

→ 观察代码法 → 递归+迭代

动态规划

Dynamic Programming

memoization (一一八)

tabulation

part.1. 暴力递归 Memoization

fibonacci(n): 1, 1, 2, 3, 5, 8 ....

n: 1 2 3 4 5 6 7 ...

1) 定义递归函数

O(2^n) time (二叉树节点数)
O(n) space (二叉树高度)

2) 记忆化递归 memo 当成 fib(n-1)，就是直接调用

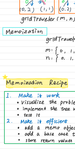
HashMap

memo
{ 1 → 2,
4 → 5,
5 → 5,
6 → 8 }

O(n) time
O(n) space

二、gridTraveler

从m*n的格子 → 右下 (不能向下或向左移动)



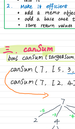
O(2^{m+n}) time
O(m+n) space

gridTraveler(m, n) = gridTraveler(n, m)

memoization

gridTraveler(4, 3)

m { 1 → 1, 2 → 3 }
n { 1 → 1, 2 → 3 }



O(m*n) time
O(m+n) space

Memoization Recipe

1. make it work
   - visualize the problem as a tree
   - implement the tree using recursion
   - test it
2. make it efficient
   - add a memo object
   - add a base case to return memo values
   - store return values into the memo

三、canSum

bool canSum(targetSum, numbers)

canSum(7, [5, 3, 4, 7]) → true

canSum(7, [2, 4]) → false

m: target sum
n: array length

O(n^m) time
O(m) space

memoized

MEMO!!! 打！

O(m*n) time
O(m) space

四、howSum

int[] howSum(target, nums)

howSum([7, [5,3,4,7]]) → [4,3] or [7]
howSum([8, [2,3,5]]) → [2,3,3] or [2,3,3] or [3,5]
howSum([7, [2, 4]]) → null
howSum([0, [2,4,5]]) → []

m: target sum
n: nums.len

O(n^m) time
O(m) space

memoized

O(n*m) time
O(m) space

五、bestSum

在测试集合中找出长度最小的数组

bestSum([8, [2,3,5]]) → [3, 5]

m: target sum
n: numbers.length

brute force:
O(n^m) time
O(m^2) space

memoized

O(n*m^2) time
O(m*n) space

Dynamic Programming

function → Decision Problem
function → Combinatoric Problem
function → Optimization Problem

六、canConstruct

canConstruct(target, wordBank)

canConstruct("abcdef", ["ab", "abc", "cd", "def", "abcd"]) → true
canConstruct("skateboard", ["bo", "rd", "ate", "t", "ska", "sk", "boar"]) → false
canConstruct("", [cat, dog]) → true

m: target length
n: wordbank.length

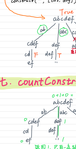
O(m)time
O(m*n) space

memoized

O(n*m^2) time
O(m^2) space

七、countConstruct

countConstruct(target, wordBank) 返回能拼出数



时间复杂度：n countConstruct !

空间：m 最深递归树的 height

八、allConstruct

allConstruct(target, wordBank) 返回能拼出所有组合

allConstruct("purple", ["purp", "p", "ur", "le", "purpl"]) → [["purp", "le"], ["p", "ur", "p", "le"]]
allConstruct("hello", ["cat", "dog"]) → []
allConstruct("", ["cat", "dog"]) → [[]]

这部分这个上可用

Part.2. tabulation

一、Fib

f[6]=8

O(n)time
O(n)space

```
for (int i=0, i<n, i++)
    f[i]=0 → f[0],
    f[i]=1 → f[1],
    f[i]=i → f[i]=f[i-1]+f[i-2]
}
```

二、gridTraveler

grid_Traveler(3,3)=6

遍历一维数组，找出每格最多四条路的长度为多少

O(m*n) time
O(m*n) space

```
for(int i=0, i<m, i++)
for(int j=0, j<n, j++)
    table[i+1][j] += table[i][j],
    table[i][j+1]+= table[i][j]
}
```

Tabulation Recipe

- visual the problem as a table
- size the table based on the inputs
- initialize the table with default values
- seed the trivial answer into the table
- iterate through the table
- fill further position based on the current position

三、canSum

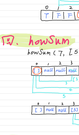
canSum(7, [5, 3, 4])→true

m: targetSum
n: nums.length

O(m)time
O(m) space

四、howSum

howSum(7, [5, 3, 4])→[3, 4]



② 和以前的题目 (target+1)

O(m*n) time
O(m*) space

五、bestSum

bestSum([8, [2, 3, 5]] → [3, 5]

O(m*n) time
O(m*) space

六、canConstruct

canConstruct(abcdef, [ab, abc, cd, def, abcd]) → true

target
wordbank

table length = target.length+1

index: 0 → ""
1 → a
2 → ab
3 → abc
4 → abcd
5 → abcde
6 → abcdef

can("", [-1]) → true

m: target
n: wordbank length

O(m*n) time
O(m) space

七、countConstruct

countConstruct("purple", ["purp", "p", "ur", "le", "purpl"]) → 2

O(m*n) time
O(m) space

八、allConstruct

all("abcdef", ["ab", "abc", "cd", "def", "abcd", "ef"])
→ [["ab", "cd", "ef"], ["ab", "c", "def"], ["abc", "def"], ["abcd", "ef"]]

时间复杂度：
O(n^m) time
O(n^m) space